

Complements of Operational Research

3.4 Sequencing problems: *branch and bound*

José Firmino Aguilar Madeira

The idea of *branch and bound*

Enumerating the $n!$ sequences guarantees the optimum, but is impractical. **Branch and bound** (B&B, Land and Doig, 1960) enumerates **implicitly**: it discards whole groups of sequences without computing them.

- **Branch**: fix the job in one position. Each node is a partial sequence; each leaf, a complete sequence.
- **Bound**: for each node, a **lower bound** on the value of *any* sequence that starts (or ends) that way.
- **Prune**: if a node's bound is $\geq$ that of the best sequence found so far (the **incumbent**), nothing below it can be better.

Pruning rule

Prune the node $\iff$ bound of the node $\geq$ value of the incumbent.

Before there is an incumbent, **nothing is pruned**.

Which node to explore next? In this course: the one with the **smallest bound** (*best-first*).

Two applications:

1. the laminating week of 3.2 (3 machines, total time);
2. one machine with due dates (total tardiness).

Part 1: three machines — the lower bound

Partial sequence σ (the first jobs), already with completion times C_1, C_2, C_3 on each machine; $U =$ jobs still to be sequenced. No sequence starting with σ ends before

$$LB_1 = C_1 + \sum_U M_1 + \min_U (M_2 + M_3),$$

$$LB_2 = C_2 + \sum_U M_2 + \min_U M_3,$$

$$LB_3 = C_3 + \sum_U M_3,$$

and the bound of the node is

$$LB = \max(LB_1, LB_2, LB_3).$$

On each machine: what has already gone through + what it still has to do + the minimum that the last job still has to go through after it.

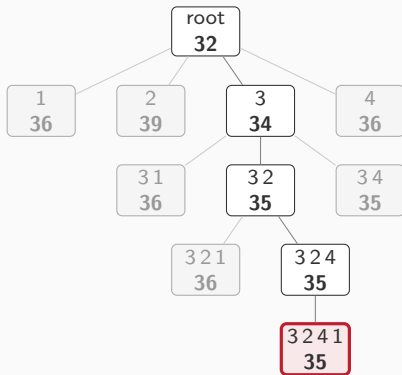
Job	Print.	Lamin.	Finish.
1	2	4	3
2	7	8	8
3	2	9	7
4	4	8	5

Node "3 first": $C = (2, 11, 18)$, $U = \{1, 2, 4\}$.

- $LB_1 = 2 + 13 + \min(7, 16, 13) = 22$
- $LB_2 = 11 + 20 + \min(3, 8, 5) = 34$
- $LB_3 = 18 + 16 = 34$

$LB = 34$: no sequence starting with 3 has $T < 34$.

The tree: optimum 3, 2, 4, 1 with $T = 35$



Order of exploration (smallest bound first):

1. root $\rightarrow$ 1: 36, 2: 39, **3: 34**, 4: 36;
2. "3" $\rightarrow$ 31: 36, **32: 35**, 34: 35;
3. "32" $\rightarrow$ 321: 36, **324: 35**;
4. "324" $\rightarrow$ 3241: **$T = 35$** , incumbent.

All the nodes still to be explored have bound $\geq 35 \rightarrow$ **pruned** (in grey). Node 34 has bound 35 = incumbent: it is pruned too — at most it would tie.

10 nodes generated, against the 24 sequences of brute force.

The print shop story comes to a close

Laminating week	Sequence	T (h)	Guarantee
Order of arrival	1, 2, 3, 4	39	—
Johnson on (G, H) (condition fails)	1, 3, 4, 2	39	none
TDM on (G, H)	2, 3, 4, 1	40	none
<i>Branch and bound</i>	3, 2, 4, 1	35	optimum

4 h less in a laminating week: the length of one shift. B&B gives the certainty that there is nothing better — which no heuristic gives.

The chapter in one table

- 2 machines: Johnson (optimal, n steps).
- m machines with the condition: Johnson on (G, H) (optimal).
- Without the condition: heuristics (Johnson, TDM) or B&B (optimal).

Part 2: one machine with due dates

Today the **digital printer** has four urgent jobs. Each one has a processing time p_j and a due date d_j (hours from 8 a.m.).

Job	p_j (h)	d_j (h)
K	2	11
L	7	8
M	3	6
N	3	4

Tardiness of job j : $T_j = \max(0, C_j - d_j)$.

Goal: minimise the total tardiness $\sum_j T_j$.

Now the total time is always $2 + 7 + 3 + 3 = 15$ h; what changes is *who* is late.

Natural heuristic — earliest due date first (EDD): N, M, L, K.

	N	M	L	K
C_j	3	6	13	15
T_j	0	0	5	4

total tardiness = 9

Is it the best? The problem is NP-hard in general; with 4 jobs there are 24 sequences.

The bound: building the sequence from the end

First the **last** job is fixed, then the second to last, ...

The last job always ends at $\sum p_j = 15$; the second to last at $15 - p_{\text{last}}$; and so on. The tardiness of the jobs already fixed at the end is therefore **known**.

Lower bound

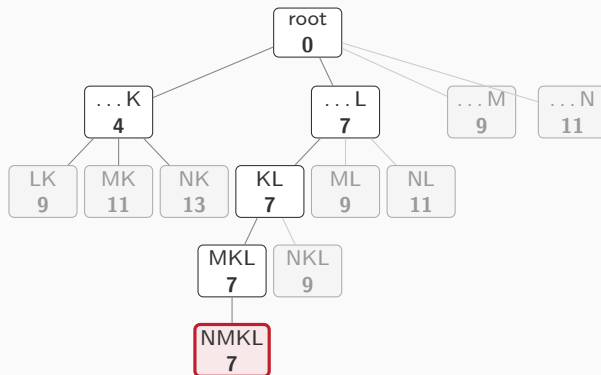
Bound of the node = sum of the tardiness of the jobs already fixed at the end.

(The others have tardiness ≥ 0 .)

Examples

- K at the end: $C_K = 15$,
 $T_K = 15 - 11 = 4 \rightarrow$ bound 4.
- L at the end: $T_L = 15 - 8 = 7 \rightarrow 7$.
- ... L, K (K last, L second to last):
 $C_L = 15 - 2 = 13$, $T_L = 13 - 8 = 5$;
bound $4 + 5 = 9$.
- ... K, L: $C_K = 15 - 7 = 8 \leq 11$,
 $T_K = 0$; bound $7 + 0 = 7$.

The tree: optimum N, M, K, L with tardiness 7



1. Root $\rightarrow$ **K**: 4, L: 7, M: 9, N: 11.
2. "... K" looks the best, but its children give 9, 11, 13.
3. "... L" (7) is explored $\rightarrow$ **KL**: 7, ML: 9, NL: 11.
4. "KL" $\rightarrow$ **MKL**: 7, NKL: 9.
5. "MKL" $\rightarrow$ NMKL: tardiness 7, incumbent.

Everything else has bound ≥ 7 : pruned.

Optimum: N, M, K, L with tardiness 7 (only L is late, by 7 h). EDD gave 9.

Node "KL": K second to last, L last.

Common mistakes

Pruning too early

A node is only pruned when its bound is $\geq$ the value of a **complete** sequence already computed. Pruning a node because “there is another one with a smaller bound” is wrong: the smaller bound may not be confirmed — that is what happened with “... K” (bound 4, but nothing below 9).

A bound that is not a bound

The bound must be $\leq$ the value of **all** the sequences in the branch. If it overestimates, the optimum may be pruned.

Ties

Bound = incumbent: it may be pruned (at most it ties). If the question asks for *all* the optimal sequences, only nodes with bound $>$ incumbent are pruned.

Confirm

At the end, recompute the value of the sequence found directly from the table (here: $C = (3, 6, 8, 15)$, tardiness 0, 0, 0, 7).

How much is saved? Problems with 8 jobs

With 8 jobs there are $8! = 40\,320$ sequences and the complete tree has 109 600 nodes. In 100 random problems, B&B (*best-first*, with the bounds of these classes) generated:

	median	90%	maximum
3 machines, total time	43	1257	5773
1 machine, total tardiness	123	432	1231

nodes generated until the optimum is proved

- In general the saving is **large**: typically hundreds of nodes instead of tens of thousands.
- But **there is no guarantee**: in the worst case B&B can come close to complete enumeration.
- Better (tighter) bounds prune more; it is also common to start with the incumbent from a heuristic (Johnson, EDD).

Exercise sheet 3 — Exercise 1 (homework)

Total tardiness, three jobs (homework)

Determine, by the *branch and bound* method, the sequence of jobs that minimises the total tardiness in the completion of all the jobs.

Job	Processing time (days)	Due date (days from today)
A	4	6
B	5	13
C	7	8

Submission: by email to the lecturer, by the deadline given in class. Exercises 2–4 (homework) and those from tests and exams: **Exercise sheet 3**.

KEY TAKEAWAYS — 3.4

1. B&B: branch (fix positions), bound (lower bound per node), prune (bound $\geq$ incumbent).
2. Three machines: $LB = \max_i (C_i + \sum_U M_i + \min_U(\text{what remains after } i))$, fixing from the start.
3. One machine, total tardiness: fix from the end; bound = tardiness of the jobs already fixed.
4. Explore the node with the smallest bound; **never prune without an incumbent**; confirm the final value.
5. B&B gives the **optimum** with a guarantee; heuristics do not (in the example: 35 h against 39 and 40 h).

End of chapter 3. Next: 4 Replacement theory.

References and resources

References Full list: *Course bibliography*.

- Land, A. H., Doig, A. G., An automatic method of solving discrete programming problems, *Econometrica*, 28(3), 497–520, 1960.
- Ignall, E., Schrage, L., Application of the branch and bound technique to some flow-shop scheduling problems, *Operations Research*, 13(3), 400–412, 1965.
- Hillier, F. S., Lieberman, G. J., *Introduction to Operations Research*, 11th ed., McGraw-Hill, 2021 — *branch and bound*.
- Pinedo, M. L., *Scheduling: Theory, Algorithms, and Systems*, 5th ed., Springer, 2016 — total tardiness on one machine.

Resources

- Exercise sheet 3. `code/python/03_scheduling/03_4_branch_and_bound/bb_sequenciamento.py` (functions `bb_fs` and `bb_atraso`).